

Papaya Recognition through Computer Vision

Advisor: Ángel Mario Lerma Sánchez

Student: German Torres Sierra

PROBLEM STATEMENT

Papaya is a tropical fruit with high demand in the market. The harvesting process depends on its ripeness and size, as these characteristics determine an approximate profit for the papaya producer.

The main issue lies in the selection and harvesting process, which is currently based on human perception. This inevitably leads to selection errors, resulting in wasted time, effort, and resources. For instance, papayas may be harvested before reaching their optimal ripeness, causing them to go to waste.

To prevent this, the creation and use of papaya recognition software is proposed. This software would determine which fruits are ready to be harvested based on their color, length, and width. As a result, the producer would only collect fruits in optimal condition, making the process more efficient.

METHODOLOGY

The main tasks expected to be executed by the software are the following:

- Individually recognize papayas through video (frames) and images, directly from the tree where they grow.
- Count the number of papayas on the tree.
- Determine their ripeness level based on color.
- Obtain their volume based on height and width.

The first step is to find algorithms and methods capable of achieving these tasks. This requires reviewing information about the current state of the art in papaya recognition.

After extensive research through academic papers and various sources, it became clear that computer vision methods and classification algorithms are necessary to meet the objectives.

Computer vision (also known as machine vision) refers to a set of processes that enable software to interpret images and emulate human visual perception.

For the most essential task—individual papaya recognition—**Convolutional Neural Networks (CNNs)** were selected.

CNNs consist of multiple layers, each with several neurons. Each neuron processes different input data and passes the result to the next layer, gradually learning until the desired output is achieved. Each connection between neurons has an assigned **weight**, and each neuron has a **bias**. These values are typically initialized randomly. Depending on whether the task involves regression or classification, and on the

dataset size, different architectures and numbers of layers or neurons can be used. However, more layers do not always guarantee higher accuracy.

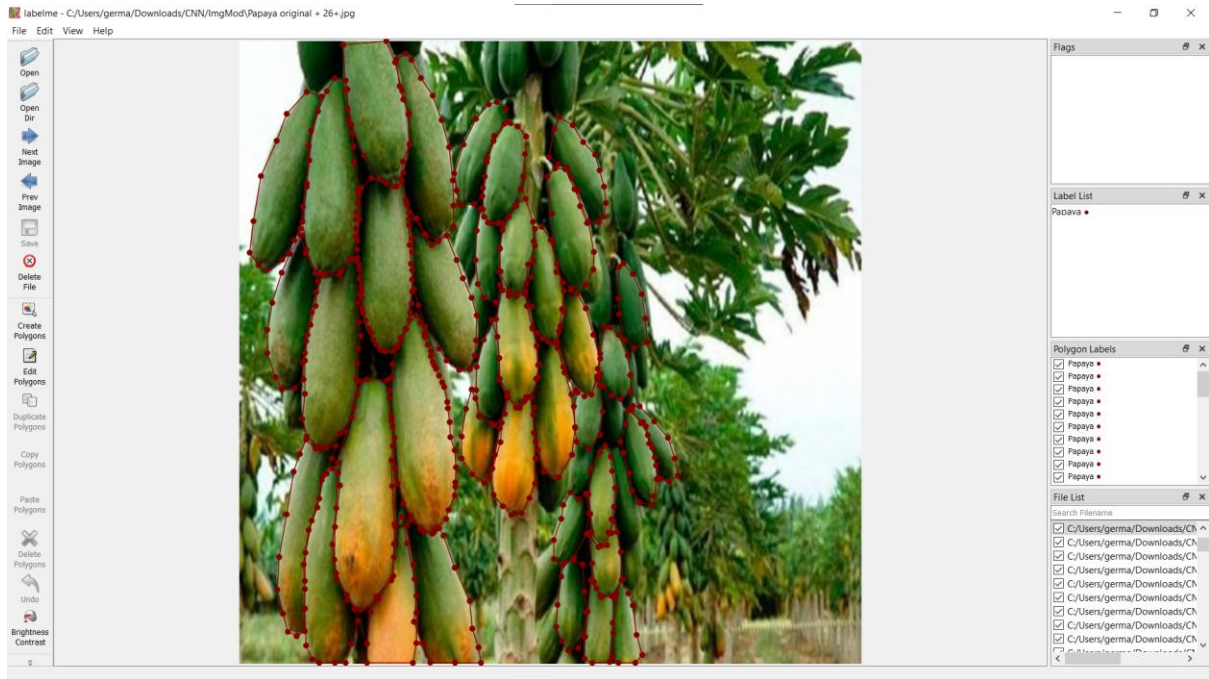
Python was chosen as the programming language, as it offers libraries and tools necessary for CNN training.

The first step for any classification algorithm is **data acquisition**—collecting images that serve as input for the model. A **dataset** is created, followed by **segmentation** to isolate relevant pixels from the rest of the image. Next comes **feature extraction**, where useful information from the region of interest is gathered for the **classification** stage, where the software learns from the extracted data.

A large number of images were collected online, including both individual papayas and papayas on trees. A **supervised learning** approach was used, meaning the data for model training were pre-labeled—so the neural network already knew what to recognize and classify.

To make the dataset more robust, the **Data Augmentation** technique was applied. This involves generating new images from existing ones by applying transformations such as zooming, cropping, or rotating, allowing the network to learn from more varied data without collecting new images.

For image labeling, the **Labelme** tool was used, allowing manual annotation of the image areas containing papayas.



A total of **140 images** were used, with **130 manually labeled**, divided as follows:

- 97 for training,
- 33 for validation,
- 10 for testing (unlabeled images).

It's crucial that test images are not used during training to accurately measure the CNN's performance on unseen data.

For CNN model creation, the **Ultralytics YOLOv8** framework was used. This framework includes several pre-trained neural network models based on the COCO dataset. Each model offers different training speeds and accuracy levels, often at the cost of greater hardware requirements. To avoid such limitations, training was performed on **Google Colab**, which provides cloud GPUs—significantly reducing training time. The **YOLOv8x** model, the most powerful one, was chosen for this reason. Model execution after training was done using **PyCharm**.

This approach applies **Transfer Learning**, where an already trained model is adapted to a new task using a custom dataset.

To train the model, several parameters were defined:

- **Epochs** – number of times the model iterates through the dataset.
- **Batch size** – number of images processed together per iteration.
- **Task** – YOLOv8 offers several options; in this case, **segment** (object segmentation) was used.

- **Image size** – YOLO requires 640×640 pixel images, so resizing was included during Data Augmentation.

Once all parameters were set, the training began.

Several training sessions were carried out until achieving the desired results. It was important to carefully select the number of epochs and batch size to avoid **overfitting** (model performs well on training data but poorly on new data) or **underfitting** (model fails to learn). The dataset size also impacts results.

- The **first training** used **20 epochs** and a **batch size of 2**, allowing the model to recognize individual papayas in specific cases.
- The **second training** used **50 epochs** and the same batch size, enabling the model to recognize both individual papayas and those on trees.

```
from ultralytics import YOLO

model = YOLO('content/yolov8x-seg.pt')
model.train(data='content/dataset.yaml', epochs=50, imgsz=640, task='segment', batch=2)
```

26/50	3.89G	1.574	2.646	1.823	1.868	83	640	100%	49/49	[00:12:00:00, 3.82it/s]	9/9	[00:01:00:00, 5.85it/s]
Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		
all	33	83	0.347	0.554	0.385	0.214	0.368	0.597	0.393	0.176		
Epoch	GPU mem	box_loss	seg_loss	cls_loss	dfl_loss	Instances	Size					
27/50	3.9G	1.549	2.715	1.803	1.828	7	640	100%	49/49	[00:12:00:00, 3.85it/s]	9/9	[00:01:00:00, 5.83it/s]
Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		
all	33	83	0.328	0.506	0.422	0.227	0.377	0.554	0.445	0.185		
Epoch	GPU mem	box_loss	seg_loss	cls_loss	dfl_loss	Instances	Size					
28/50	4.08G	1.568	2.592	1.736	1.874	9	640	100%	49/49	[00:12:00:00, 3.78it/s]	9/9	[00:01:00:00, 5.72it/s]
Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		
all	33	83	0.31	0.566	0.429	0.206	0.31	0.627	0.427	0.173		
Epoch	GPU mem	box_loss	seg_loss	cls_loss	dfl_loss	Instances	Size					
29/50	4.02G	1.538	2.434	1.679	1.85	3	640	100%	49/49	[00:12:00:00, 3.85it/s]	9/9	[00:01:00:00, 5.88it/s]
Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		
all	33	83	0.352	0.59	0.45	0.223	0.424	0.554	0.469	0.186		
Epoch	GPU mem	box_loss	seg_loss	cls_loss	dfl_loss	Instances	Size					
30/50	3.86G	1.467	2.405	1.742	1.799	26	640	100%	49/49	[00:12:00:00, 3.88it/s]	9/9	[00:01:00:00, 5.94it/s]
Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	Mask(P	R	mAP50	mAP50-95): 100%		
all	33	83	0.551	0.548	0.487	0.226	0.55	0.529	0.496	0.224		



CONCLUSIONS

Throughout the internship, valuable feedback was received from the advisor, who had significant experience with the chosen method. This made the learning process enriching and satisfying.

In the end, one of the main goals was to achieve **individual papaya detection on trees**. Although this represents the basic stage of the project, it opens the door to future development.

I believe I've only scratched the surface of the vast field of computer vision, but I feel proud of the results accomplished.